

Pengenalan Stored Procedure

ISA212 Basisdata 2

Kuliah 4

Mira Musrini

Benefits of using a stored procedure

- It can be easily modified: We can easily modify the code inside the stored procedure without the need to restart or deploying the application. For example, If the T-SQL queries are written in the application and if we need to change the logic, we must change the code in the application and re-deploy it. SQL Server Stored procedures eliminate such challenges by storing the code in the database. so, when we want to change the logic inside the procedure we can just do it by simple ALTER PROCEDURE statement.

Benefits of using a stored procedure

- Reduced network traffic: When we use stored procedures instead of writing T-SQL queries at the application level, only the procedure name is passed over the network instead of the whole T-SQL code.
- Reusable: Stored procedures can be executed by multiple users or multiple client applications without the need of writing the code again.

Benefits of using a stored procedure

- **Security:** Stored procedures reduce the threat by eliminating direct access to the tables. we can also encrypt the stored procedures while creating them so that source code inside the stored procedure is not visible. Use third-party tools like ApexSQL Decrypt to decrypt the encrypted stored procedures.
- **Performance:** The SQL Server stored procedure when executed for the first time creates a plan and stores it in the buffer pool so that the plan can be reused when it executes next time.

EXERCISE

```
CREATE TABLE Product
(ProductID INT, ProductName VARCHAR(100) )
GO
```

```
CREATE TABLE ProductDescription
(ProductID INT, ProductDescription VARCHAR(800) )
GO
```

```
INSERT INTO Product VALUES (680,'HL Road Frame - Black, 58') , (706,'HL Road
Frame - Red, 58')
, (707,'Sport-100 Helmet, Red')
GO
```

```
INSERT INTO ProductDescription VALUES (680,'Replacement mountain wheel for
entry-level rider.') , (706,'Sturdy alloy features a quick-release hub.')
, (707,'Aerodynamic rims for smooth riding.')
GO
```

Creating a simple stored procedure

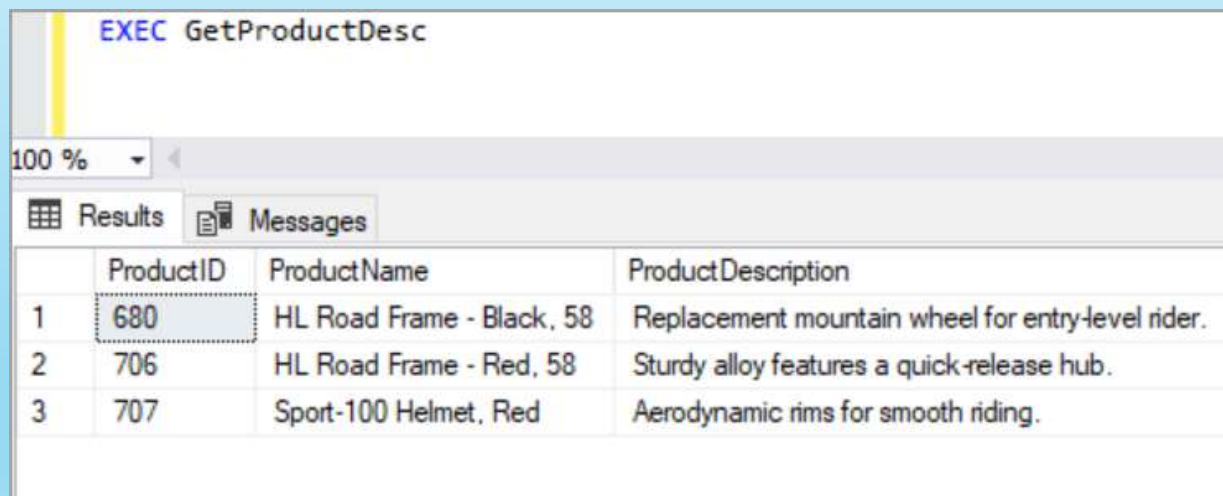
```
CREATE PROCEDURE GetProductDesc
AS
BEGIN
SET NOCOUNT ON

SELECT P.ProductID,P.ProductName,PD.ProductDescription FROM
Product P
INNER JOIN ProductDescription PD ON P.ProductID=PD.ProductID

END
```

Execution Stored Procedure

We can use 'EXEC ProcedureName' to execute stored procedures. When we execute the procedure GetProductDesc, the result set looks like below.



EXEC GetProductDesc

100 %

Results Messages

	ProductID	ProductName	ProductDescription
1	680	HL Road Frame - Black, 58	Replacement mountain wheel for entry-level rider.
2	706	HL Road Frame - Red, 58	Sturdy alloy features a quick-release hub.
3	707	Sport-100 Helmet, Red	Aerodynamic rims for smooth riding.

Creating a stored procedure with parameters

Following is the example of a stored procedure that accepts the parameter.

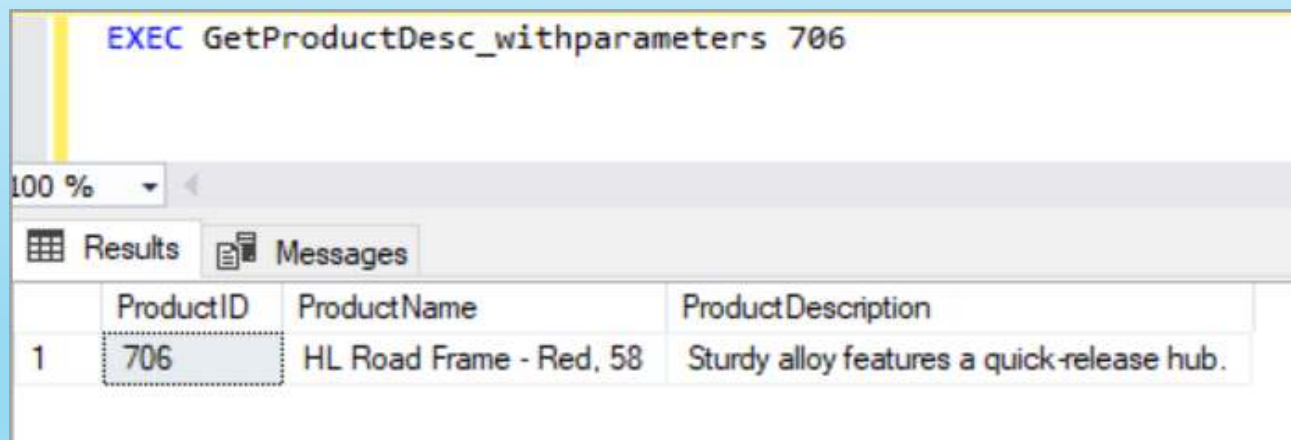
```
CREATE PROCEDURE GetProductDesc_withparameters
(@PID INT)
AS
BEGIN
    SET NOCOUNT ON

    SELECT P.ProductID,P.ProductName,PD.ProductDescription FROM
    Product P
    INNER JOIN ProductDescription PD ON P.ProductID=PD.ProductID
    WHERE P.ProductID=@PID
END
```

Executing stored procedure with input parameter

```
EXEC GetProductDesc_withparameters 706
```

While executing the stored procedure we need to pass the input parameter. Please refer to the below image for the result set.



The screenshot shows a SQL Server Enterprise Manager window with the command `EXEC GetProductDesc_withparameters 706` entered in the query editor. Below the editor, the 'Results' tab is selected, displaying a table with the following data:

	ProductID	ProductName	ProductDescription
1	706	HL Road Frame - Red, 58	Sturdy alloy features a quick-release hub.

Creating a stored procedure with default parameters

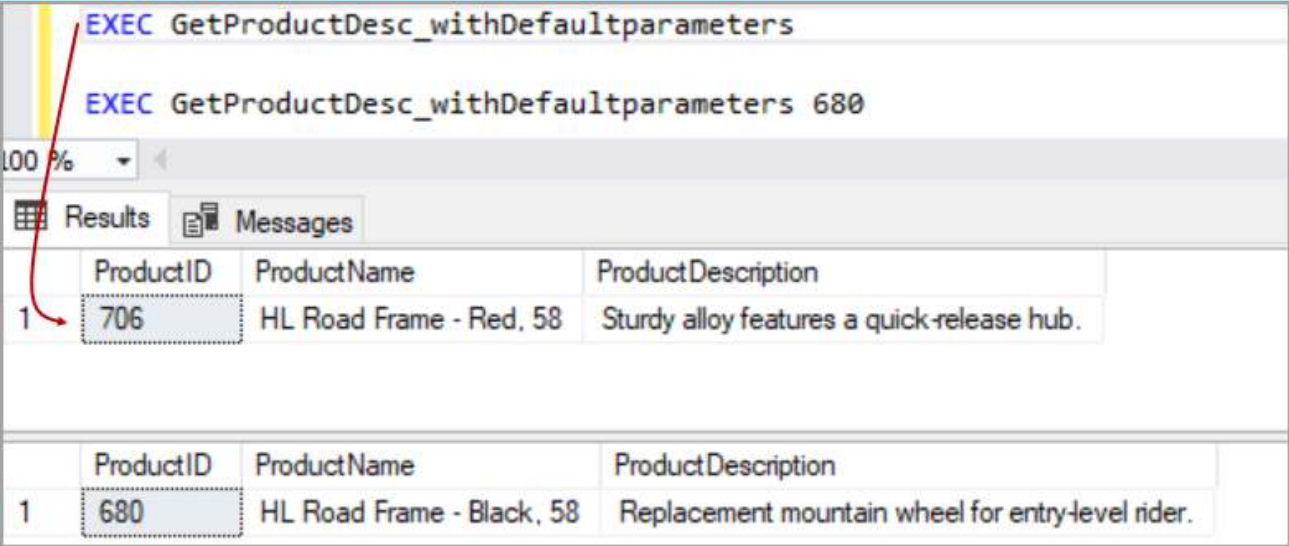
```
CREATE PROCEDURE GetProductDesc_withDefaultparameters
(@PID INT =706)
AS
BEGIN
SET NOCOUNT ON

SELECT P.ProductID,P.ProductName,PD.ProductDescription FROM
Product P
INNER JOIN ProductDescription PD ON P.ProductID=PD.ProductID
WHERE P.ProductID=@PID

END
```

Executing a stored procedure with default parameters

- When we execute the above procedure without passing the parameter value, the default value 706 will be used. But when executed passing the value, the default value will be ignored and the passed value will be considered as a parameter.



The screenshot shows the SQL Server Enterprise Manager interface. At the top, two execution commands are listed: `EXEC GetProductDesc_withDefaultparameters` and `EXEC GetProductDesc_withDefaultparameters 680`. Below these, the 'Results' tab is selected, displaying two tables of results. The first table corresponds to the first command (no parameters), showing a result for ProductID 706. The second table corresponds to the second command (with parameter 680), showing a result for ProductID 680. A red arrow points from the 'EXEC' command to the 'Results' tab.

	ProductID	ProductName	ProductDescription
1	706	HL Road Frame - Red, 58	Sturdy alloy features a quick-release hub.

	ProductID	ProductName	ProductDescription
1	680	HL Road Frame - Black, 58	Replacement mountain wheel for entry-level rider.

Creating a stored procedure with an output parameter

- Below is the example of a stored procedure with an output parameter. The following example retrieves the EmpID which is an auto identity column when a new employee is inserted.

```
CREATE TABLE Employee (EmpID int identity(1,1), EmpName varchar(500))
Go
CREATE PROCEDURE ins_NewEmp_with_outputparamaters
(@Ename varchar(50), @EId int output)
AS
BEGIN
    SET NOCOUNT ON
    INSERT INTO Employee (EmpName) VALUES (@Ename)
    SELECT @EId= SCOPE_IDENTITY()
END
```

Executing a stored procedure with an output parameter

- Executing the stored procedures with output parameters is bit different. We must declare the variable to store the value returned by the output parameter.

```
declare @EmpID INT
EXEC ins_NewEmp_with_outputparamaters 'Andrew', @EmpID
OUTPUT
SELECT @EmpID
```

result

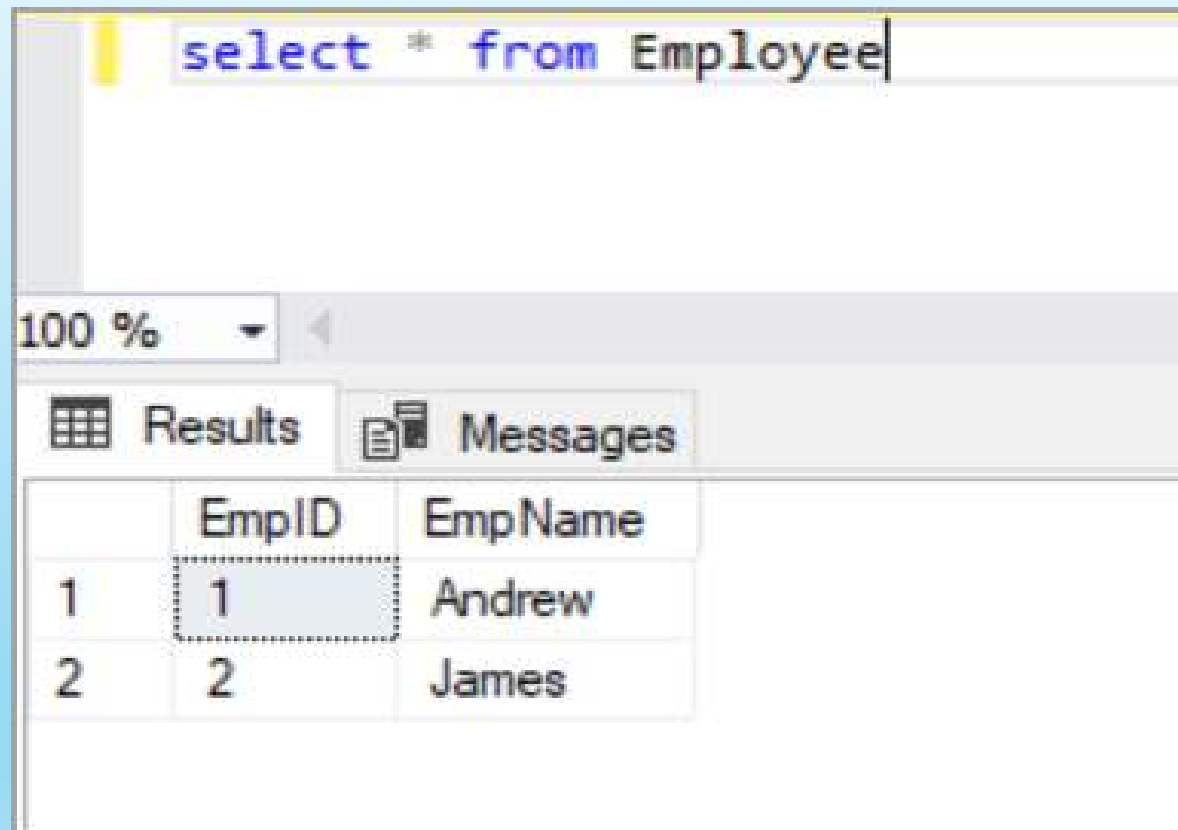
```
declare @EmpID INT  
  
EXEC ins_NewEmp_with_outputparamaters 'Andrew', @EmpID OUTPUT  
  
SELECT @EmpID as EmployeeID  
  
declare @EmpID2 INT  
  
EXEC ins_NewEmp_with_outputparamaters 'James', @EmpID2 OUTPUT  
  
SELECT @EmpID2 as EmployeeID
```

100 %

Results Messages

	EmployeeID
1	1
1	2

Records of employee table



The screenshot shows a database query interface. At the top, a text box contains the SQL query: `select * from Employee`. Below the text box is a zoom level dropdown set to "100 %". Underneath are two tabs: "Results" (active) and "Messages". The "Results" tab displays a table with two columns: "EmpID" and "EmpName". The table contains two rows of data. The first row has "1" in the "EmpID" column and "Andrew" in the "EmpName" column. The second row has "2" in the "EmpID" column and "James" in the "EmpName" column. The cell containing "1" in the first row is highlighted with a dashed border.

	EmpID	EmpName
1	1	Andrew
2	2	James

SP_HELPTEXT <store procedure's name>

We can type `Sp_helptext <store procedure's name>` to print the source code of the store procedure.

For Example : `sp_helptext GetProductDesc`

Creating an encrypted stored procedure

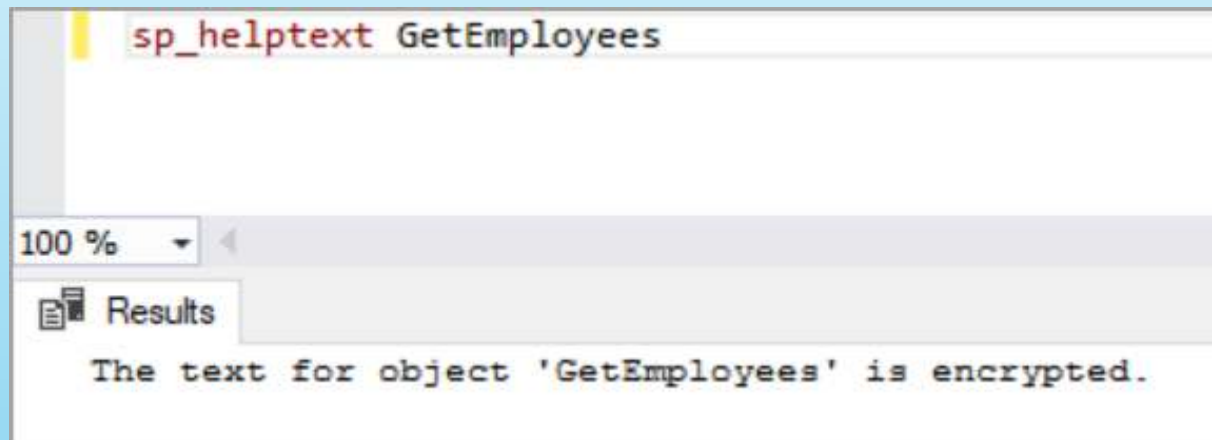
We can hide the source code in the stored procedure by creating the procedure with the “ENCRYPTION” option.

Following is the example of an encrypted stored procedure.

```
CREATE PROCEDURE GetEmployees  
WITH ENCRYPTION  
AS  
BEGIN  
    SET NOCOUNT ON  
    SELECT EmpID, EmpName from Employee  
END
```

Creating an encrypted stored procedure

- When we try to view the code of the SQL Server stored procedure using `sp_helptext`, it returns “The text for object ‘GetEmployees’ is encrypted.”



Creating a temporary procedure

- Like the temporary table, we can create temporary procedures as well. There are two types of temporary procedures, one is a local temporary stored procedure and another one is a global temporary procedure.
- These procedures are created in the tempdb database.

LOKAL TEMPORARY PROCEDURE

- Local temporary SQL Server stored procedures: These are created with # as prefix and can be accessed only in the session where it created. This procedure is automatically dropped when the connection is closed.
- Following is the example of creating a local temporary procedure.

```
CREATE PROCEDURE #Temp  
AS  
BEGIN  
PRINT 'Local temp procedure'  
END
```

GLOBAL TEMPORARY PROCEDURE

- Global temporary SQL Server stored procedure: These procedures are created with ## as prefix and can be accessed on the other sessions as well. This procedure is automatically dropped when the connection which is used to create the procedure is closed.
- Below is the example of creating a global temporary procedure.

```
CREATE PROCEDURE ##TEMP  
AS  
BEGIN  
PRINT 'Global temp procedure'  
END
```

Modifying the stored procedure

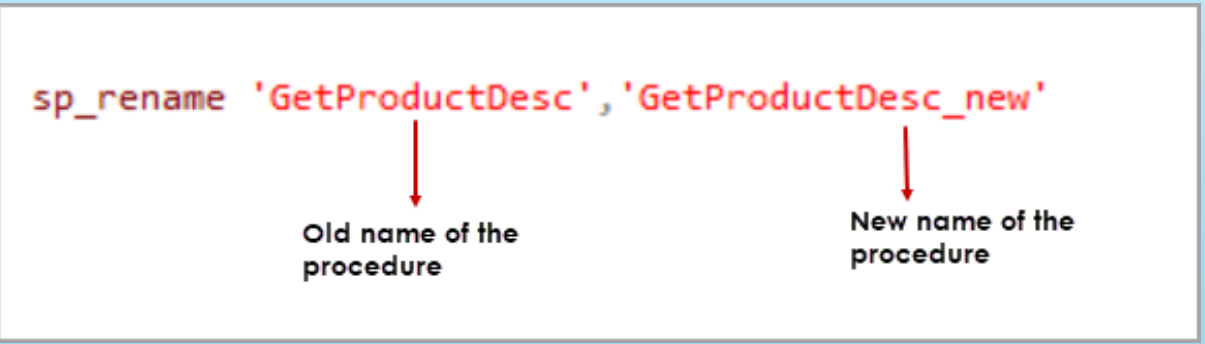
- Use the ALTER PROCEDURE statement to modify the existing stored procedure. Following is the example of modifying the existing procedure.

```
ALTER PROCEDURE GetProductDesc
AS
BEGIN
    SET NOCOUNT ON
    SELECT P.ProductID,P.ProductName,PD.ProductDescription FROM
    Product P
    INNER JOIN ProductDescription PD ON P.ProductID=PD.ProductID
END
```

Renaming the stored procedure

- To rename a stored procedure using T-SQL, use system stored procedure `sp_rename`. Following is the example that renames the procedure “GetProductDesc” to a new name “GetProductDesc_new”.

```
sp_rename 'GetProductDesc', 'GetProductDesc_new'
```



```
sp_rename 'GetProductDesc', 'GetProductDesc_new'
```

Old name of the
procedure

New name of the
procedure